Uniform Domain Randomization and Residual Policy Learning in Mujoco Hopper Environment

Luca Genca

***Abstract*—This work explores reinforcement learning strategies for improving sim-to-real transfer using the MuJoCo Hopper environment. In the first stage, I trained the Hopper to walk using Proximal Policy Optimization (PPO) from Stable-Baselines3. To make the learned policy more robust, I applied Uniform Domain Randomization (UDR), introducing variations in the Hopper's mass during training. To test generalization, I evaluated the policy in an environment where the torso mass was increased by 1kg, simulating a reality gap and seeing if it could handle the unseen conditions.**

Building on this, I introduced a more challenging task: jumping over obstacles. To help the Hopper adapt, I used Residual Policy Learning, allowing it to refine its behavior beyond what the base policy had learned. I compared performance across different settings: testing the Hopper with and without residual learning, as well as with and without UDR—to see how each method influenced its ability to generalize. Additionally, I expanded the Hopper's observation space to include obstacle positions, which helped it navigate varying obstacle layouts more effectively.

I. Introduction

One of the biggest challenges in Reinforcement Learning (RL) is the gap between simulation and real-world performance, often referred to as the "sim-to-real" transfer problem. Policies trained in simulation sometimes struggle when deployed in real-world environments due to discrepancies in sensor accuracy, environmental conditions, and other unpredictable factors. In this work, I focus on addressing this issue by improving policy robustness through domain randomization techniques.

For my experiments, I used the MuJoCo Hopper environment, which is a standard benchmark in RL research. The goal was to train an agent to walk using the Proximal Policy Optimization (PPO) algorithm from Stable-Baselines3. The main objective was to study how well policies trained in a simulated environment adapt to changes. To create a "reality gap," I shifted the torso mass by +1kg between the training and testing environments.

To improve the policy's generalization ability, I applied Uniform Domain Randomization (UDR), which exposes the policy to a range of variations during training, encouraging it to perform well under different conditions. Then, I introduced a more complex task for the Hopper: jumping over obstacles. To help the Hopper with this task, I incorporated Residual Policy Learning (RPL), which refines the learned base policy to learn new, more complex behaviors.

Additionally, I expanded the observation space to include the positions of obstacles in the environment, enabling the Hopper to better navigate more complex and dynamic layouts. I then compared the performance across different setups: with and without residual learning, and with and without UDR.

II. Background

Reinforcement learning has made impressive progress in enabling agents to perform tasks like walking, running, and navigating complex environments. Algorithms such as Proximal Policy Optimization (PPO) have demonstrated great success, but transferring these learned policies from simulation to real-world scenarios still presents a significant challenge. To bridge this "reality gap," methods like domain randomization and residual policy learning have been introduced.

Domain Randomization (DR) improves the generalization of policies by exposing the agent to a variety of training conditions. By randomizing physical parameters like mass, DR ensures that the learned policy is less sensitive to specific environmental conditions, making it more adaptable to unseen situations. A specific variant of DR, known as Uniform Domain Randomization (UDR), uses a uniform distribution as a probability distribution to sample the parameters. Mathematically, DR modifies the simulation by sampling environment parameters p from a predefined distribution $P(p)$, thus creating a wider range of conditions for training.

Residual Policy Learning (RPL) is used to learn more complex tasks by adding a residual policy to an already trained base policy. The base policy handles fundamental actions, like walking, while the residual policy refines these actions to suit specific tasks. The combined policy is:

$$\pi_{\text{combined}}(a_t|s_t) = \pi_{\text{base}}(a_t|s_t) + \pi_{\text{residual}}(a_t|s_t),$$

where π_{base} represents the base action, and π_{residual} adjusts this action based on task-specific observations. This approach allows the base policy to maintain its core functionality while the residual policy adapts to new challenges.

In this work, I integrate UDR and RPL within the MuJoCo Hopper environment, with a focus on two primary tasks: adapting to variations in torso mass and navigating obstacles.

III. Related Works

Residual Policy Learning (RPL) is a technique introduced by Silver et al. for improving imperfect policies using deep reinforcement learning. The core idea behind RPL is to learn a residual function that augments an existing base policy. This approach is beneficial even when the original controller is non-differentiable. Silver et al. applied RPL to several MuJoCo tasks that involved complex robotic manipulation, where starting from scratch would be inefficient or impractical.

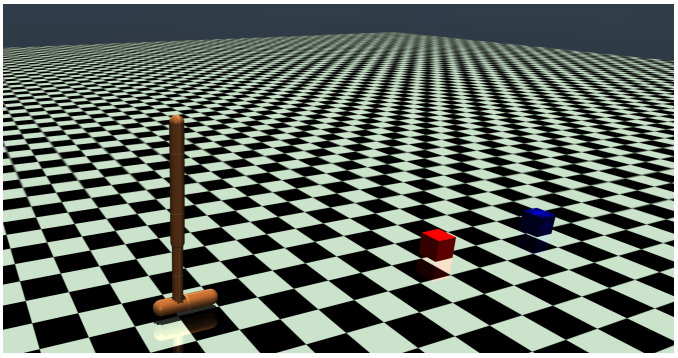


Fig. 1: Image of the modified environment with the obstacles

Their results showed that RPL consistently improves the performance of initial policies, while also being far more data-efficient than traditional reinforcement learning approaches.

The study also explores the effectiveness of RPL in scenarios involving partial observability, sensor noise, model inaccuracies, and miscalibrated controllers. The findings suggest that RPL outperforms learning from scratch.

IV. Methodology

For the experiments, I chose the MuJoCo Hopper environment as the testbed. This environment provides a continuous observation space with 11 dimensions, where each state variable can take any value in the range $(-\infty, +\infty)$.

Num	Observation	Min	Max	Joint	Unit
0	Height of hopper	-Inf	Inf	rootz	m
1	Angle of the top	-Inf	Inf	rooty	rad
2	Thigh joint angle	-Inf	Inf	thigh_joint	rad
3	Leg joint angle	-Inf	Inf	leg_joint	rad
4	Foot joint angle	-Inf	Inf	foot_joint	rad
5	Velocity of x-coord	-Inf	Inf	rootx	m/s
6	Velocity of height	-Inf	Inf	rootz	m/s
7	Angular vel. of top	-Inf	Inf	rooty	rad/s
8	Angular vel. of thigh	-Inf	Inf	thigh_joint	rad/s
9	Angular vel. of leg	-Inf	Inf	leg_joint	rad/s
10	Angular vel. of foot	-Inf	Inf	foot_joint	rad/s

TABLE I: Observation space details for the MuJoCo Hopper.

The action space is also continuous, consisting of three dimensions, with each action restricted to the range $[-1, 1]$.

Num	Action	Min	Max	Unit
0	Torque on thigh rotor	-1	1	N·m
1	Torque on leg rotor	-1	1	N·m
2	Torque on foot rotor	-1	1	N·m

TABLE II: Action space details for the MuJoCo Hopper.

These control inputs affect the movement of the Hopper's four masses: the torso (3.53 kg), thigh (3.93 kg), leg (2.71 kg), and foot (5.09 kg).

To simulate a "reality gap," I altered the environment by reducing the torso mass by 1 kg in the source environment compared to the target environment. For the reinforcement learning algorithm, I used Proximal Policy Optimization (PPO) from Stable-Baselines3.

To study the impact of Uniform Domain Randomization (UDR) on the Hopper's performance, I conducted several training and testing experiments. I first trained the Hopper in the source environment and tested it in both the source and target environments. I then repeated this by training the Hopper in the target environment, both with and without UDR. The results, including the mean and standard deviation of rewards, are summarized in Table III.

To further examine the effect of UDR in more complex settings, I modified the environment by adding two obstacles in the form of boxes. The positions of these obstacles along the x-axis were different in the source and target environments (2 and 4 for the source environment, and 2.5 and 3.8 for the test environment), increasing the task complexity.

I then ran experiments to investigate how UDR impacted the Hopper's performance under these altered conditions. As before, I trained the Hopper in the source environment and tested it in both the source and target environments, with and without UDR. The results, including mean rewards and standard deviations, are shown in Table V.

To enhance the Hopper's performance, I decided to add a residual policy on top of the base policy. The goal of this residual policy was to fine-tune the actions of the base policy without negatively impacting its initial behavior. To do this, I initially set the last layer of the action network to zero and set its standard deviation to 4.54×10^{-5}. This ensured that, at the start of training, the residual policy didn't contribute to the action, so the behavior was identical to that of the base policy. This approach ensured that the addition of the residual policy wouldn't worsen the performance of the base policy during the early stages of training. The base policy used in this setup was the one trained in the environment without obstacles.

Additionally, I froze the last layer of the action network during the initial phase of training. This allowed the value network to focus on quickly exploiting the base policy, accelerating learning. Only once the critic loss dropped below 1 did I unfreeze the action network, ensuring that updates from the residual policy would be more effective and based on a solid foundation. The results of this approach are shown in Table VI.

After some experimentation, I realized that the information available to the Hopper might not be enough for it to navigate the obstacles effectively. To address this, I expanded the observation space to include the positions of the obstacles. This modification has important implications for real-world applications: for a robot to use such an algorithm, the terrain would need to be scouted beforehand, and the positions of the obstacles would need to be registered on the robot. This seemed like a reasonable assumption for a robot tasked with jumping over obstacles.

With the expanded observation space, I ran the tests again

using both PPO and the residual policy. The results, including the mean rewards and standard deviations, are presented in Table VII.

Finally, to test how well the Hopper could generalize across different obstacle layouts, I randomized the positions of the two obstacles during each testing episode. This experiment was conducted using both PPO and PRL, and the results are summarized in Table VIII.

V. Results

Configuration	Training→Testing	Mean Reward +/- Std Dev
No UDR	Source→Target	845.5265504 ±37.0169
	Source→Source	1596.15015104 ±1.5564
	Target→Target	1065.34859294 ±61.5922
UDR	Source→Target	1107.77682824 ±131.5131
	Source→Source	1623.68277942 ±220.6055
	Target→Target	1773.27724488 ±31.7759

TABLE III: Performance of the Hopper with and without UDR under different training and testing configurations.

Range of Mass Variations	Mean Reward ± Std Dev
50%-150%	1107.78 ± 131.51
90%-110%	901.03 ± 53.77
10%-190%	1007.48 ± 61.50

TABLE IV: Mean reward and standard deviation for different mass variation ranges.

Algorithm	Training→Testing	Mean Reward +/- Std Dev
PPO (No UDR)	Source → Target	673.03935424 ±15.5071
	Source→Source	728.3419368 ±101.1920
PPO (UDR)	Source→Target	818.51351486 ±7.1931
	Source→Source	1075.98969724 ±89.2092

TABLE V: Performance of PPO with and without UDR in the obstacle environment.

Algorithm	Training→Testing	Mean Reward +/- Std Dev
PRL (No UDR)	Source→Target	897.96709296 ±100.5210
	Source→Source	1099.47611692 ±215.5224
PRL (UDR)	Source→Target	910.9722782 ±156.2233
	Source→Source	605.51170152 ±3.3146

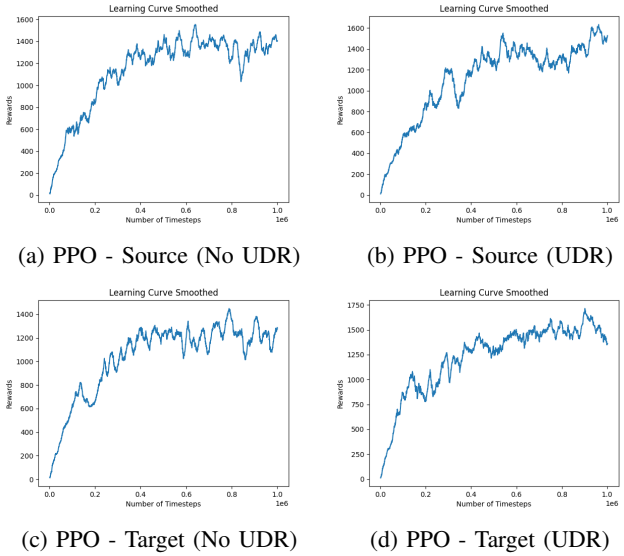
TABLE VI: Performance of PRL with and without UDR in the obstacle environment..

Algorithm	Training→Testing	Mean Reward +/- Std Dev
PPO	Source→Target	947.62994144 ±35.1811
	Source→Source	605.37906654 ±2.5386
PRL	Source→Target	946.33549964 ±133.3446
	Source→Source	776.27564798 ±74.1349

TABLE VII: Performance of PPO and PRL with expanded observation space including obstacle positions.

Algorithm	Source → Target (Mean Reward ± Std Dev)
PPO	737.84223184 ± 95.65497186
PRL	1008.28266406 ± 334.74937546

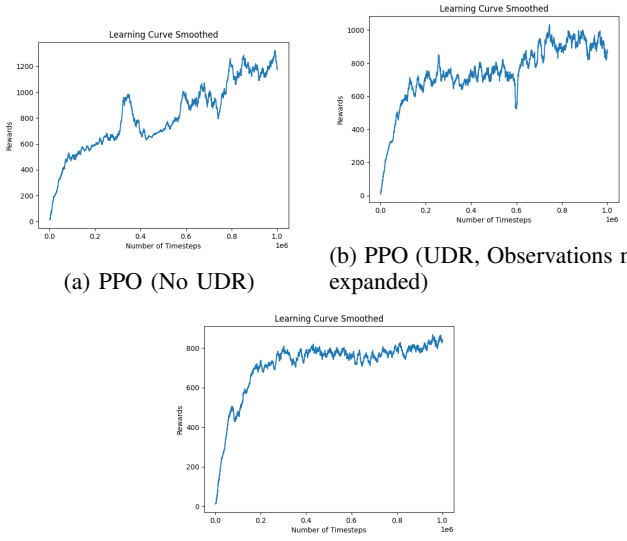
TABLE VIII: Performance of the Hopper with randomized obstacle positions during testing.



(a) PPO - Source (No UDR) (b) PPO - Source (UDR)

(c) PPO - Target (No UDR) (d) PPO - Target (UDR)

Fig. 2: Training performance of PPO in different environments, with and without UDR.

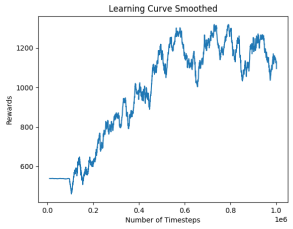


(a) PPO (No UDR)

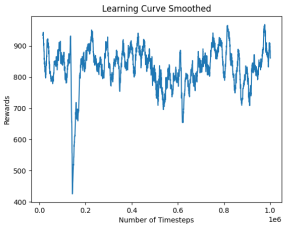
(b) PPO (UDR, Observations not expanded)

(c) PPO (UDR, Observations expanded)

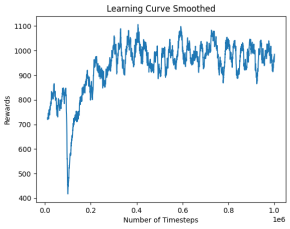
Fig. 3: Training performance of PPO in the environment with obstacles.



(a) PRL (No UDR)



(b) PRL (UDR, Observations not expanded)



(c) PRL(UDR, Observations expanded)

Fig. 4: Training performance of PRL in the environment with obstacles.

VI. Analysis of Results

As we can see from Table III, performance in the Source→Target test, is lower when compared to the Target→Target test, because training directly in the test environment lets the policy learn the environment better. This however is not feasible in reality because of the potential harm that training in the real world brings to the robot and because of time constraints (training is much slower in the real world when compared to simulation).

UDR showed an improvement over the naive source→target configuration, but a limitation of UDR is the sensibility to the randomization range: too narrow of a range and the policy won't learn to generalize, too wide of a range and the environment gets unnecessarily complex for the policy (Table IV).

Regarding RPL, we can see an improvement in both training time (by looking at figures 3a and 4a we can see that PRL reached its peak around 500000 timestamps, while PPO took 1000000 timestamps) and performance (Table V and VI). Utilizing UDR in this new environment with obstacles proved to be a bit of a challenge for both algorithms (in particular PRL where i had to increase the threshold for the critic loss to 200 because the critic loss was not stabilizing and never went lower than around 100) and this can be seen in figures 3b and 4b.The latter in particular shows no improvement over the base policy.

This is probably because the hopper has no knowledge of where the obstacles are and doesn't understand why the same sequence of states lead to vastly different rewards. This is why i added the position of the obstacles to the observation space and, as can be seen in figure 4c, it did improve performance. The improvement observed in the training did not translate that well in the testing environment, as we can see from table VII, because neither algorithm was able to consistently jump over the second obstacle and thus they both ended up obtaining similar results. However by randomizing the position of the obstacles during the test (Table VIII) we can see that PRL is able to perform better than PPO indicating that it is able to generalize better.

VII. Future Works

An interesting idea that was not investigated in this work is the possibility of using different kinds of networks as residual network. In particular it would be interesting to try and use a CNN to process the images from a camera mounted on the hopper and use that to generate the residual action.

References

[1] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual Policy Learning," arXiv preprint arXiv:1812.06298, Jan. 2019. [Online]. Available: https://doi.org/10.48550/arXiv.1812.06298